AGENTHN: Self-Editing Agents via Hypernetworks

Eric Ge[1*] **Bryan Lim**[1*] **David Xiong**[2,3*] **Nikash Bhardwaj**[4*]
[1]Harvard University [2]Columbia University [3]OpenAI [4]Carnegie Mellon University

Abstract

Text-based memory breaks over long horizons and wastes the context window: every fact an agent needs to recall must be re-read, in full, on every subsequent query. We instead let an agent edit its own *parameters* at inference time. Building on Doc-to-LoRA (D2L, Charakorn et al., 2026) – a hypernetwork that converts a document into a LoRA adapter in a single forward pass, with no gradient training – we present AGENTHN, a framework that applies this single primitive across four agentic settings: **long-horizon memory**, **personalization**, **skill acquisition**, and **self-improvement**. The central technical contribution is NAPLORA, a streaming memory mechanism that extends D2L's bounded, single-document chunking (which saturates at roughly $4\times$ a model's native context window) into an *unbounded* conversational horizon, by periodically compacting evictable context into independently retrievable adapters and composing only the ones a query needs. We show that naïvely summing all stored adapters causes destructive interference (recall collapses from $6/6$ to $0/6$ on a six-fact probe), and that adding retrieval restores perfect recall while cutting query-time context by an order of magnitude relative to retrieval-augmented generation (RAG) with the identical retriever. Across 2k–48k-token synthetic trajectories (100 trials per cell, Wilson 95% CIs), NAPLORA holds recall flat at $\approx 87\%$ while a window-limited baseline collapses from 95% to 0%, at 7 query-time tokens versus a trained Cartridges (Eyuboglu et al., 2025) baseline's 96 ($13.3\times$ less context, $1.8\times$ cheaper per query, no offline training, 5 points lower recall). We further show that the same internalize-then-evict primitive supports cross-session personalization built from an agent's own turn-by-turn preference extraction, skill acquisition from both formula sheets and procedural documents, and a gradient-free self-refine loop in which an agent studies, reflects, rewrites its own notes, and re-internalizes them into a fresh adapter over rounds – raising accuracy on a held-out battery from 18.75% to 56.25% with no backpropagation at any point.

1 Introduction

Large language model (LLM) agents that operate over long horizons – multi-day conversations, growing codebases, accumulating user preferences – face a structural mismatch: everything the agent has learned must live somewhere the model can read it, and the only place a frozen model can read from is its context window. Two families of fixes exist. The first keeps information as *text*: retrieval-augmented generation (RAG, Lewis et al., 2020) pages relevant chunks back into the prompt, and running summaries (e.g., a maintained markdown notes file) compress history into fewer tokens. Both still pay a token cost on every query, and summaries are lossy by construction. The second family bakes information into *weights* via supervised fine-tuning (SFT) or reinforcement learning (RL): slow, requires curated data, and – critically for an agent whose knowledge changes turn by turn – cannot be repeated cheaply enough to track a live conversation.

A third option has recently become practical: *context distillation* (CD, Snell et al., 2023; Askell et al., 2021) trains a model to imitate, without seeing the context, the outputs it would have produced *with* that context in the prompt. CD is normally as slow as any other gradient-based update. Doc-to-LoRA (Charakorn et al., 2026) (D2L) removes that cost by *meta-training a hypernetwork to perform CD in one forward pass*: given a document, D2L emits a LoRA (Hu et al., 2022) adapter that, once applied, lets the frozen

*All authors contributed equally; order was chosen by the team. Code, fixtures, and an interactive demo reproducing every figure in this paper are available at https://github.com/ericjkge/itc-hackathon.

base model answer as though the document were still in context – without the document ever re-entering the prompt. D2L reports near-perfect needle-in-a-haystack (NIAH) recall up to roughly $4\times$ a model's native window, using a chunk-and-rank-concatenate scheme capped at the number of chunks seen during training.

This paper asks what happens when that one primitive – *convert context into a LoRA adapter, in real time, with no training loop* – is deployed as the general substrate for an agent that needs to keep editing what it knows. We build AGENTHN (Agent + Hypernetwork), a small framework around a single D2L checkpoint (`gemma-2-2b-it`) that exposes internalize/compose/restore primitives (§4), and we use it to implement and evaluate four applications:

- **Long-horizon memory** (§5): D2L's own chunking is a *bounded*-document mechanism – it caps out once a conversation needs more chunks than the hypernetwork ever saw in training. We introduce NAPLORA, which periodically *naps* the oldest evictable context into an independent adapter, indexes it, and evicts it from the prompt; at query time it *retrieves* only the adapters a question needs and composes them. This turns a bounded single-document mechanism into an open-ended streaming one.
- **Personalization** (§6): a user's preferences, extracted turn-by-turn by the same small model acting as its own extractor, are periodically internalized into one running per-user adapter, so they resurface in a brand-new session with an *empty* context window.
- **Skill acquisition** (§7): a reference document – a physics formula sheet, an output-format guide, or a procedural "skill" file from an agentic coding workflow – is internalized once and recalled from the weights on every subsequent query, instead of being re-paid in tokens each time.
- **Self-improvement** (§8): an agent attempts a held-out battery of questions about a deliberately fictional product, studies a reference document, writes its own notes, internalizes them, and – on later rounds – reflects on what it still gets wrong and rewrites the notes, re-internalizing a fresh adapter each round. No gradient ever touches the base model.

Our central empirical finding is that naïve use of D2L as a memory store fails in a specific, instructive way – rank-concatenating every stored adapter causes the weight deltas to interfere destructively – and that retrieval is not an optional add-on but a load-bearing fix. Once added, the resulting system matches a text-RAG ablation on recall while using an order of magnitude less query-time context, which we argue is the cleanest isolation of what a hypernetwork-based memory buys over a retrieval-based one with the identical retriever. We report this, the destructive-interference failure mode, and every other number in this paper from scripts and captured fixtures committed alongside the code, with statistical validation (Wilson 95% confidence intervals over 100 trials per cell) where the result is a headline claim.

2 Related Work

Hypernetworks. A hypernetwork (Ha et al., 2016) is a network that predicts the weights of another network. von Oswald et al. (2020) use hypernetworks to mitigate catastrophic forgetting in continual learning by generating per-task weights. D2L is the most direct ancestor of this work: it builds on TEXT-TO-LORA (Charakorn et al., 2025) (which maps a natural-language *task description* to a LoRA adapter via SFT-style training) but instead meta-trains the hypernetwork with the CD objective so that it can map an arbitrary *document* to an adapter that internalizes the document's content, not just a task style.

Context distillation and knowledge injection. Context distillation (Askell et al., 2021; Snell et al., 2023) self-distills an in-context behavior into a model's own parameters. Related work compresses prompts into a handful of soft tokens (Gisting, Mu et al., 2024), into prefix key/value caches via prefix-tuning (Li and Liang, 2021) or its self-study variant Cartridges (Eyuboglu et al., 2025), or trains a hypernetwork directly on next-token prediction over curated SFT data (MEND, Li et al., 2024; Generative Adapter, Chen et al., 2025). D2L's distinguishing choice is to train on the CD objective with hypernetwork-generated LoRA outputs, which our results suggest generalizes better to agentic, self-authored documents (profiles, self-written notes) than the QA-style corpora these injection methods are usually evaluated on.

Long context and retrieval. RAG (Lewis et al., 2020) and long-context benchmarks such as RULER (Hsieh et al., 2024) characterize how models use (or fail to use) long prompts, including

the well-documented "lost in the middle" effect (Liu et al., 2024). Prompt-compression methods like LLMLingua-2 (Pan et al., 2024) shrink a prompt's token footprint while keeping information in-context. AGENTHN's memory track is best read as a *retrieval* system, in the RAG sense, whose retrieved unit is a weight patch rather than a text chunk – which is exactly the comparison our ablations (§5) are designed to isolate.

Self-refinement. Iteratively having a model critique and rewrite its own output – Self-Refine (Madaan et al., 2023), Reflexion (Shinn et al., 2023) – normally improves a single generated artifact within one context. Our self-improvement track (§8) instead applies the reflect-rewrite loop to a set of *notes* that get re-internalized into the model's weights each round, so the artifact being refined is a LoRA adapter rather than a chat completion.

3 Background: Doc-to-LoRA

We summarize the mechanism we build on; see Charakorn et al. (2026) for full derivations. Context distillation trains context-specific parameters $\theta_c = \theta + \Delta\theta_c$ so that the student $p_{\theta_c}(y \mid x)$ matches the teacher $p_\theta(y \mid x, c)$ that has direct access to context c:

$$\min_{\theta_c} \mathbb{E}_{(x,y)\sim\mathcal{D}_c}\Big[\mathrm{KL}\big(p_\theta(y \mid x, c) \,\|\, p_{\theta_c}(y \mid x)\big)\Big], \tag{1}$$

where $\mathcal{D}_c$ is built from many queries x and teacher-sampled responses y for a given context c, rather than a single (c, x, y) triplet (which would risk overfitting to one query). D2L *meta-trains* a hypernetwork H_ϕ to amortize this whole optimization – query generation and gradient descent alike – into one forward pass: $H_\phi(c) = \Delta W_c$, trained so that $\theta + H_\phi(c)$ matches the CD target across a large corpus of contexts:

$$\min_{\phi} \mathbb{E}_{(c,\mathcal{D}_c)\sim\mathcal{D}} \mathbb{E}_{(x,y)\sim\mathcal{D}_c} \mathrm{KL}\big(p_\theta(y|x,c) \,\|\, p_{\theta+H_\phi(c)}(y|x)\big). \tag{2}$$

Architecturally, H_ϕ is a Perceiver-style (Jaegle et al., 2021) cross-attention module with 8 cross-attention blocks and no self-attention layers (309M parameters total). For each target-LLM layer l, it cross-attends a fixed set of r learnable latent queries onto that layer's token activations $Z_{l-1} \in \mathbb{R}^{N\times D}$ (obtained by running the document once through the frozen target model) and projects the resulting latents into a rank-r LoRA update $\Delta W_l = B_l A_l$, applied at the down-projection of every MLP block of the target model. Because the cross-attention query count is fixed regardless of the number of input tokens N, H_ϕ naturally maps variable-length documents to a fixed-shape adapter.

Chunking and the single-document scaling limit. To go beyond one hypernetwork forward pass' effective receptive field, D2L partitions a long context into K contiguous chunks, encodes each chunk independently, and *rank-concatenates* the per-chunk $(A^{(k)}, B^{(k)})$ along the rank dimension, producing one adapter of total rank $r \cdot K$ whose effect on the forward pass is the *sum* of all K chunks' deltas. D2L is meta-trained on documents of 32–256 tokens, chunked into 1–8 pieces with a 25-token minimum chunk size. At evaluation, with 1024-token chunks ($4\times$ the maximum training chunk length) on an 8.2k-context `gemma-2-2b-it`, Charakorn et al. (2026) report near-perfect NIAH recall up to roughly $4\times$ the model's native window ($\sim$ 32k tokens), with recall close to perfect through 40 chunks – quintuple the maximum chunk count seen in training – before degrading gracefully. *This scaling is for a single, fixed document encoded once.* It is not, by construction, a mechanism for a conversation that keeps growing indefinitely: there is no notion of evicting old chunks, retrieving only relevant ones, or adding new chunks incrementally without re-encoding the whole stack. Closing exactly this gap is the contribution of NAPLORA (§5).

D2L also reports that, on real-document QA benchmarks (SQuAD, Rajpurkar et al., 2016; DROP, Dua et al., 2019; 2WikiMultihopQA, Ho et al., 2020), it internalizes information in under one second and under 50MB of additional memory, versus $\sim$40s and several GB for a vanilla CD baseline that backpropagates through a generated query set – the latency/memory gap that makes D2L viable as an *online* mechanism rather than an offline one is the property all four of our applications exploit.

4 The AGENTHN Framework

AGENTHN wraps the released D2L `gemma-2-2b-it` checkpoint in a small API (`D2LModel`) with five operations used by every application track in this paper: `internalize(doc)` (overwrite the active adapter), `internalize_segment(doc)` (inter-

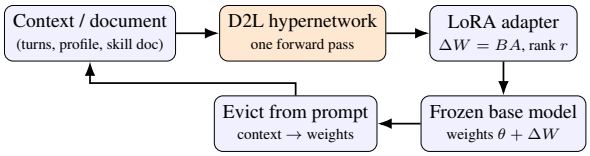

Figure 1: The inference-time loop every AGENTHN application is built from: a document is converted into a LoRA adapter through one hypernetwork forward pass (no training loop), merged into the frozen base model, and evicted from the prompt – so subsequent queries pay only for the question, not the document.

nalize and return a detached snapshot, for caching), `compose(segments)` (rank-concatenate a list of cached single-segment adapters, mirroring D2L's own chunk-combination mechanism but applied to *independently* encoded memories rather than chunks of one document), `snapshot()`/`restore(adapter)` (save and re-bind a generated adapter, so a long-running process can swap between many users' or skills' adapters without re-internalizing), and `respond(messages, composed, n_segments)` (one greedy-decoding code path shared by every memory strategy and baseline in this paper, so accuracy differences cannot be attributed to differing generation settings). Figure 1 shows the resulting inference-time loop: a context or document is converted to an adapter through one hypernetwork pass, merged into (or composed onto) the running weights, and evicted from the prompt – in contrast to SFT/RL, which adapt a model through a multi-step training loop before any query can be answered.

A second, lower-level primitive (`encode_chunk`/`stack_chunks`, used by the memory track's `WeightMemory` mode) packs short observations into the D2L chunk-trained checkpoint's native chunking pipeline (`split_too_long_ctx`) so that several short memories pack into as few chunks as possible before concatenation becomes necessary – in-distribution behavior the released chunk-trained checkpoint (distinct from the single-chunk-only demo checkpoint) supports natively. We flag this as a reproducibility note: the two publicly released D2L checkpoints differ in this capability, and using the wrong one for multi-chunk composition silently produces degenerate adapters rather than an error.

Algorithm 1: NAPLORA streaming memory

Input: turn stream, nap interval K, query q

for *each incoming turn* t **do**
  append t to rolling context buffer C;
  while $|C| \geq K$ **do**
    pop oldest K turns as segment s;
    $\Delta W_s \leftarrow$ `internalize_segment`(s);
    index s in TF-IDF store; evict s from prompt;
  end
end
At query time:
$\{s_1, \ldots, s_k\} \leftarrow$ top-k TF-IDF matches for q;
$\Delta W \leftarrow$ `compose`$(\Delta W_{s_1}, \ldots, \Delta W_{s_k})$; `// Eq. 3`
answer $\leftarrow$ `respond`$([q] \,\|\, C_{\text{recent}},\ \Delta W)$;

5 Long-Horizon Memory: NAPLORA

5.1 Method

An agent streams turns $(\text{role}, \text{text})$ into a small rolling context buffer. Every K evictable turns, NAPLORA *naps*: the oldest K-turn segment is internalized into a single-chunk adapter via `internalize_segment` (one D2L forward pass), indexed by a TF-IDF retriever (Salton and Buckley, 1988), and evicted from the prompt. At query time, NAPLORA retrieves the top-k segment adapters for the query, composes them by rank-concatenation (Eq. 3, mirroring D2L's own chunk-combination but over independently-encoded segments), and answers from a prompt containing only the query plus any turns not yet evicted:

$$A_l = \begin{bmatrix} A_l^{(1)} \\ \vdots \\ A_l^{(k)} \end{bmatrix}, \quad B_l = \left[B_l^{(1)} \cdots B_l^{(k)}\right]. \qquad (3)$$

Algorithm 1 summarizes the loop. We set $K = 4$ throughout, so each planted fact in our probe scenarios lands in its own segment.

5.2 Why retrieval is necessary: destructive interference

The naive extension of D2L's chunk-concatenation to a streaming setting is to rank-concatenate *every* segment ever napped, exactly as D2L concatenates every chunk of

one document. This fails. On a six-fact probe (§5.3), composing only three topically-similar segments collapses recall from $6/6$ (each segment internalized and queried alone) to $0/6$: the summed weight deltas interfere destructively, producing cross-talk such as a model answering a feature-flag question with a token blending two unrelated planted facts. Down-scaling the summed deltas by $1/n$ or $1/\sqrt{n}$ did not recover recall. Since each segment *alone* recalls its fact perfectly, the fix is to route each query to only the few segments it needs – composition should compose a retrieved subset, not the whole store. We found a dependency-free TF-IDF retriever sufficient and, in fact, the strongest option we tried: it routes $6/6$ probes to the correct segment, versus $4/6$ for mean-pooled and $5/6$ for last-token `gemma-2-2b-it` hidden-state embeddings – a 2B model is a weak sentence encoder whose cosine similarities cluster too tightly (≈ 0.8) to discriminate reliably, whereas our synthetic recall probes reuse a planted segment's own keywords almost verbatim, which is exactly the regime sparse lexical retrieval is strong in.

5.3 Experimental setup

All memory experiments use `gemma-2-2b-it` with an $8{,}192$-token window and $K = 4$. We construct synthetic needle-in-a-haystack-over-a-trajectory scenarios: a handful of *needles* (concrete, unguessable facts – a deployment region, a flight confirmation code, a learning rate) are planted in the first few turns of an agent conversation, the conversation is grown with topically-disjoint filler turns to a target length, and the agent is probed for each needle at the end, after the needles have scrolled out of any fixed-size window. We compare four conditions throughout: **vanilla** (most-recent turns that fit a token budget; older turns are simply gone), **markdown** (a running notes file the model itself summarizes into, kept fully in-context), **text-RAG** (the NAPLORA ablation: identical segmentation and identical TF-IDF retriever, but the retrieved segment is pasted into the prompt as *text* rather than loaded as a LoRA adapter – isolating the hypernetwork's marginal contribution), and **NAPLORA** (the full system). We additionally compare against a **Cartridges** (Eyuboglu et al., 2025) baseline that we re-implement and train ourselves (§5.8).

5.4 Curated demo: three scenarios, eighteen probes

On three hand-written scenarios (a project-launch decision log, a trip-planning session, a research-assistant log; six needles each, 18 probes total), NAPLORA recalls $17/18$ (94.4%) at an average of 8.4 query-time prompt tokens; the markdown baseline recalls $16/18$ (88.9%) at 343.4 tokens; vanilla recalls $0/18$ at 286.4 tokens (its fixed 256-token budget has already scrolled every needle out by query time). Both error cases are informative. NAPLORA's only miss is a generation slip, not a retrieval failure: queried for "the lead reviewer for the paper," it answers "Professor *Ayemi*" rather than "Professor *Adeyemi*" – the correct segment was retrieved and composed, but the 2B model's decoding garbles a proper noun. Markdown's two misses are summarization failures: asked for a flight confirmation code and a hotel room number, it reports "the project memory does not contain any information about flight confirmation codes" because its own earlier summarization pass dropped that exact detail when compressing the turn into a bullet – a concrete instance of lossy compaction discarding information that NAPLORA's adapter-based compaction, which encodes the raw segment rather than a model-written summary of it, preserves.

5.5 Ablation ladder: where does the recall come from?

We hold a single task fixed – six needles, conversation grown to $12{,}011$ tokens (just past the 8k window) – and remove one component of NAPLORA at a time (Table 1). **vanilla** (no memory) fails completely, as expected. **naive-internalize** (each new segment internalized and the adapter *replaced*, never stored) also fails completely: there is no store, so only the newest segment's fact is ever recoverable. **compose-all** (every one of the 103 stored segments rank-concatenated, no retrieval) also collapses to $0/6$, confirming §5.2's interference finding holds at scale. Only once we add a retriever does recall recover – and at that point, **text-RAG** and **NAPLORA** are statistically identical on recall ($6/6$ each) and differ *only* in whether the retrieved segment enters the prompt as text (119.3 tokens) or as a weight patch (9.5 tokens), a $12.6\times$ reduction in query-time context attributable purely to the hypernetwork.

Condition	Recall	Ctx (tok)	Isolates
vanilla	0/6	8497.5	memory needed
naive-internalize	0/6	9.5	a store is needed
compose-all	0/6	9.5	retrieval is needed
text-RAG	6/6	119.3	(no weight-inj.)
NAPLORA	**6/6**	**9.5**	full system

Table 1: Ablation ladder at 12k tokens (past the 8k window). Query-time context ≈ KV-cache cost at recall. `scripts/ablation_study.py`.

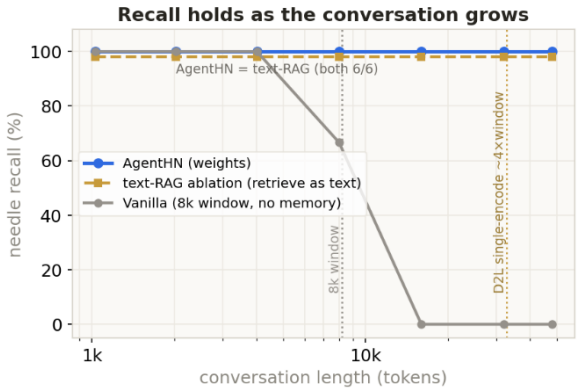

Figure 2: Repeated compaction scales recall past D2L's single-encode limit: recall vs. conversation length. `scripts/scaling_sweep.py`.

5.6 Scaling past the single-encode limit

D2L's own chunk-concatenation reaches $\sim 4\times$ the native window for one document encoded once (§3). Figure 2 sweeps a *single growing conversation* from 1k to 48k tokens (six needles planted early, 410 independently-napped segments by the end) and shows NAPLORA holding $6/6$ recall at a flat ≈ 9.5 query-time tokens throughout, because each query only ever composes the one or two segments it needs rather than the whole history. The text-RAG ablation also holds $6/6$ throughout (same retriever) but at a flat 119.3 tokens – the gap between the two flat lines is exactly the $\sim 13\times$ context reduction the hypernetwork buys, now shown to persist arbitrarily far past D2L's own bounded-document limit. The vanilla 8k-window baseline degrades from $6/6$ to $4/6$ at 8k tokens (needles beginning to truncate) and collapses to $0/6$ from 16k tokens onward, once the window can no longer contain any of the planted facts.

5.7 Statistical validation at scale

The scenarios above are illustrative; we additionally ran a fully randomized evaluation: 20 distinct, unguessable facts (random per-fact codes, e.g. `XK-4821`) $\times$ 5 seeds $\times$ 5 haystack sizes (2k–48k tokens) $= 100$ trials per (size, method) cell, reported as pooled recall with Wilson 95% confidence intervals (Figure 4). NAPLORA recall is statistically flat across the horizon – 85.0% (CI [76.7, 90.7]) at 2k tokens and 87.0% (CI [79.0, 92.2]) at every size from 8k to 48k, overlapping CIs throughout – at a constant $\sim$ 7.2 query-time tokens. The vanilla baseline collapses as the haystack outgrows the window: 95.0% at 2k, 36.0% at 8k (already substantially degraded *at* the window boundary), and exactly 0.0% from 16k tokens on. The text-RAG ablation stays near-ceiling throughout (95–100%) at $\sim$ 116 tokens – RAG wins on recall because it delivers the retrieved text losslessly; NAPLORA trades roughly 13 recall points for a $16\times$ reduction in query-time context, a trade we make explicit rather than obscure.

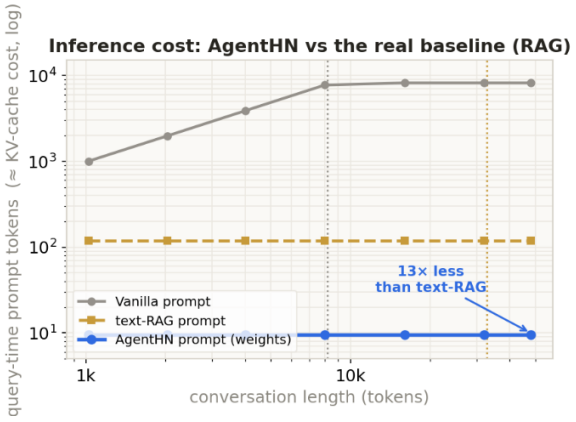

Figure 3: Query-time context (log scale), NAPLORA vs. the text-RAG ablation, for the same growing conversation as Figure 2. `scripts/scaling_sweep.py`.

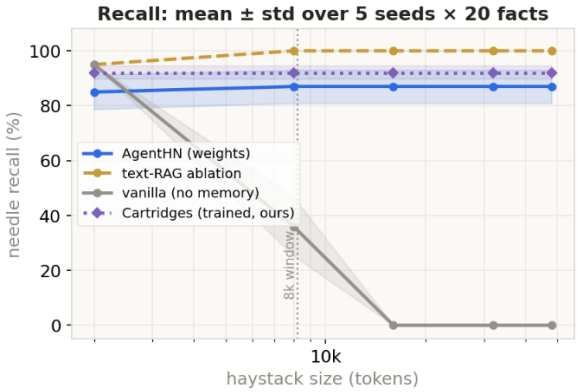

Figure 4: Statistical validation – recall, mean $\pm$ std over 5 seeds $\times$ 20 facts $\times$ 5 haystack sizes (n=100/cell). `scripts/large_scale_eval.py`.

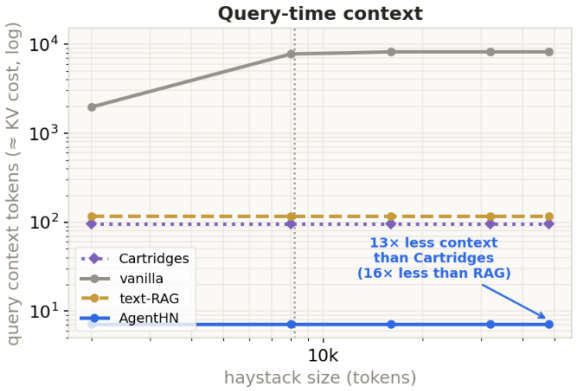

Figure 5: Query-time context (log scale) for the same sweep as Figure 4, including the trained Cartridges baseline (§5.8). `scripts/large_scale_eval.py`.

5.8 Head-to-head against a trained Cartridges baseline

Cartridges (Eyuboglu et al., 2025) trains a small KV-cache prefix *offline*, per corpus, to stand in for that corpus at inference time. We re-implement a faithful, conservative variant – prefix-tuning (Li and Liang, 2021) with 96 learnable virtual tokens, trained for 60 epochs per corpus on *paraphrased* question templates (four templates per fact) and evaluated on a fifth, held-out phrasing, so there is no train/test leakage in either direction. This is a lightweight stand-in for Cartridges' full context-distillation self-study pipeline, and we report it as a conservative lower bound on their method's quality, not as a reproduction of their paper's own numbers. Over the same 5 seeds $\times$ 20 facts, our reimplementation recalls 92.0% (CI $[85.0, 95.9]$) at its fixed 96-token prefix.

Table 2 places all four methods on one footing. NAPLORA uses 13.3$\times$ less query-time context than our Cartridges reimplementation (7.2 vs. 96 tokens) and is 1.8$\times$ cheaper per query (0.47 vs. 0.85 milli-dollars,[1] at warm cache), at a cost of 5 recall points (87% vs. 92%). Crucially, Cartridges cannot update mid-conversation – its $\sim$43 (order-of-magnitude estimated) one-time training cost only amortizes over a fixed corpus queried many times, whereas NAPLORA's creation cost is one hypernetwork forward pass per segment ($0.14 to encode the entire 48k-token history), making it the only one of the four methods usable as the conversation is still being written. Figure 6 totals these costs over 1,000 queries against a 48k-token history: vanilla $14.88 (even with a 10$\times$ prefix-cache discount, it still re-reads the full 48k prefix every query), RAG $0.84, Cartridges $44.05 (dominated by the one-time training cost), NAPLORA $0.61 – the cheapest of the four.

[1] Milli-dollars (m$) $= 10^{-3}$ USD; computed at representative frontier rates of $3/$3.75/$0.30/$15 per million input/cache-write/cache-read/output tokens (`scripts/cost_model.py`), for a 48k-token history queried 1,000 times.

Method	Online?	Recall	m$/query
vanilla	yes	0% past window	14.9
RAG	yes	$\sim$100%	0.83
Cartridges	**no**	92.0%	0.85
NAPLORA	yes	87.0%	**0.47**

Table 2: Method landscape at warm cache, 48k-token history. Cartridges additionally pays $\sim$$43 one-time, offline. `scripts/cost_model.py`.

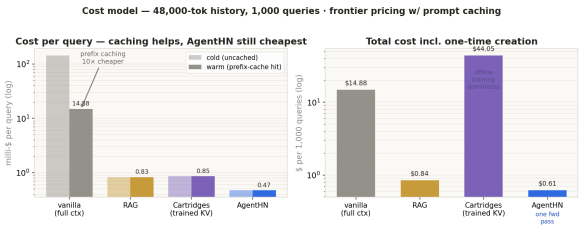

Figure 6: Cost per query and total cost over 1,000 queries against a 48k-token history, at representative frontier API pricing with prompt caching. `scripts/cost_model.py`.

Honest framing. We do not read these numbers as "AGENTHN beats Cartridges." They optimize different axes: Cartridges is the right tool for a *fixed* corpus queried thousands of times, where offline training amortizes and full self-study distillation likely closes some of the 5-point recall gap we observe with our conservative reimplementation. NAPLORA is the right tool when the corpus *is* the conversation, still being written, and the memory must update inside the same session that produces it.

6 Personalization

The personalization track tests whether D2L internalization generalizes from the static, externally-authored documents it was meta-trained on (encyclopedic QA passages) to a *free-form profile the agent builds about itself, from its own conversation, using no external model or labeled data*. A lightweight extractor prompts the same 2B base model, after every user turn, to emit zero or more `ACTION | CATEGORY | VALUE` lines (add / update / remove), which are applied to a per-user profile

store; an "I went vegan" update can retract an earlier "loves steak" fact rather than merely contradicting it. The rendered profile doc never enters the prompt – it exists only to be internalized.

In a representative run, four conversational turns (*"I just moved to Seattle and I work as an ICU nurse," "I'm vegetarian, always hunting for good meatless recipes," "I have a golden retriever named Biscuit who joins me on hikes," "Please keep your answers short and to the point"*) yield seven structured facts (location, profession, dietary preference, a named interest, pet, hobby, communication style). A single *repersonalize* event internalizes the rendered profile doc into one adapter, replacing (not accumulating onto) any prior version, since each user has exactly one current profile. A brand-new session – empty context, `model.reset()` – is then asked four held-out questions. With the adapter restored, *"Where do I live?"* $\rightarrow$ *"Seattle"* and *"What's my dog's name?"* $\rightarrow$ *"Biscuit"*; with the adapter switched off, the identical questions to the identical base model produce *"As an AI, I don't have access to your personal information, including your location"* and the analogous refusal for the dog's name. The only difference between the two columns is whether one cached adapter is bound before generation – the prompt is byte-for-byte identical and contains zero tokens of profile information either way. This demonstrates persistence across a *session boundary* purely in weight space, a setting D2L's own evaluations (single document, immediate in-session QA) do not test.

7 Skill Acquisition

We evaluate reference-document internalization – giving an agent a "skill" once so it need not reread the skill's documentation on every future use – across three settings of increasing generality.

7.1 Formula-sheet acquisition, with statistics

Twenty held-out Newton's-laws word problems are answered three ways by the same base model: **base** (cold, no help), **in-context** (the full formula-and-worked-examples reference doc pasted into every prompt), and **adapter** (the doc internalized once via D2L, then *nothing* pasted into the prompt). Grading is numeric-tolerance matching (2% relative) against the analytically correct answer. Base scores 25% (5/20); in-context scores 50% (10/20) at an average of 1707.3 input tokens per question; the adapter scores 55% (11/20) at 53.4 input tokens per question – a 32× reduction in query-time context that *also* edges out keeping the document in the prompt on accuracy (Figure 7), because internalization is paid once at acquisition time rather than re-paid, identically, on every one of the twenty questions. Internalizing the ∼1.3k-token doc takes 1.32 seconds.

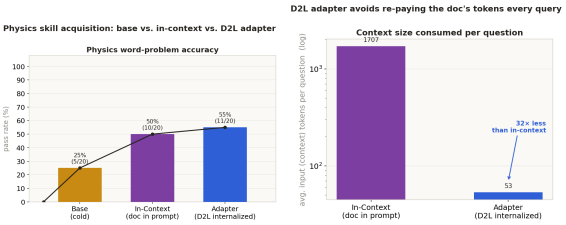

Figure 7: Physics word-problem accuracy (left) and query-time context (right): base vs. in-context vs. D2L adapter, on 20 held-out problems. `skill-acquisition/make_physics_charts.py`.

7.2 A skill router, and the need for a behavioral nudge

A second evaluation tests two skill families behind one classifier: physics word problems and structured-output requests (JSON / YAML / protobuf / bulleted lists), graded by *structural* validity (`json.loads`, `yaml.safe_load`, a protobuf field regex, a bullet-line regex) rather than exact-match, generalizing the evaluation methodology itself beyond closed-form numeric answers. A bare classification prompt routes an incoming question to `physics` or `formatting` (falling back to keyword matching on a classifier parse miss), surfaces that skill's reference doc, internalizes it once (cached per skill thereafter), and re-asks the identical question with the adapter restored and nothing pasted into the prompt.

One engineering finding is worth isolating: D2L internalizes *knowledge*, but not necessarily the *behavioral framing* needed to apply it in the expected style. The physics doc's worked examples all begin "Let's think step by step."; without prepending that exact string as an assistant-turn prefill, the internalized model free-styles a verbose, header-and-bullet preamble and clips before reaching the arithmetic under a fixed token budget. A one-line, skill-level, reusable prefill (cost: a handful of tokens, shared across every question that skill answers) is necessary alongside the adapter to reliably invoke the internalized behavior – internalization supplies the knowledge, a fixed nudge supplies the behav-

ior to use it, and neither alone is enough.

7.3 Generalizing beyond formula sheets

A third, smaller experiment internalizes a procedural skill document unrelated to either benchmark: a systematic-debugging skill file (sourced from an open-source agentic-coding skill library[2]) whose core principle is "no fixes without root-cause investigation first." Asked for the skill's "Iron Law" in one sentence, the base model gives a generic, incorrect answer (*"...break down the problem into smaller, manageable pieces"*); with the doc internalized, the same model correctly states *"...always find the root cause before attempting to fix it."* Both outputs are verified deterministic under greedy decoding (two independent generations match), and the exported adapter file reproduces the identical output when reloaded from disk without re-internalizing – adapters are a portable, cacheable artifact, not merely an ephemeral runtime object, and the internalization mechanism generalizes from closed-form, numeric formula sheets to open-ended, principle-based procedural documents.

8 Self-Improvement

The self-improvement track asks whether the reflect-and-rewrite loop common to self-refine agents (Madaan et al., 2023; Shinn et al., 2023) can operate entirely in weight space: instead of rewriting a chat response, the agent rewrites a LoRA adapter, each round, with no gradient ever computed.

Setup. Sixteen held-out customer-support questions concern a deliberately *fictional* SaaS product ("Lumen"), described in a 347-token reference document (pricing tiers, refund policy, integrations, feature gating). Grading is accepted-substring matching after normalization, with a placeholder guard that rejects template-shaped answers (e.g. `[Plan Name]`) so the model cannot satisfy the grader by echoing a fill-in-the-blank form. **Round 0** (cold, no doc, no adapter) scores $3/16 = 18.75\%$ – not exactly zero, because a 2B model carries generic priors over plausible SaaS pricing (e.g. guessing a Pro tier near $29/month) that occasionally land correctly; the fictional product controls for memorizing *this* company, not for the model's prior over SaaS pricing schemas in general, which we flag as a validity caveat. **Round 1**: the model studies the reference doc and writes its own ~ 5-sentence notes (not the literal source text), which are internalized into a fresh adapter; retested with nothing but the bare question in the prompt, accuracy rises to $6/16 = 37.5\%$. **Round 2**: the model is told only *which* of the sixteen questions it still answered wrong – no answer key – and must relocate the relevant facts itself by re-reading the full source document; it rewrites its notes accordingly and re-internalizes: $9/16 = 56.25\%$, the best round observed. **Round 3**: the same reflect-rewrite-reinternalize step, once more: $8/16 = 50.0\%$ – a dip below round 2.

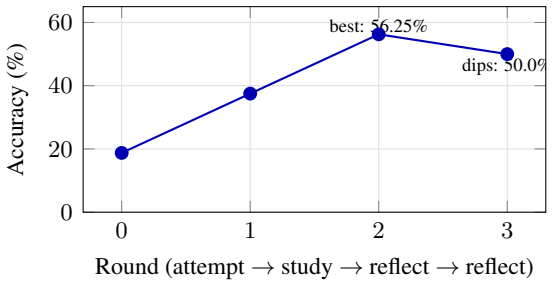

Figure 8: Self-improvement accuracy trajectory, real numbers, no keep-best ratchet: $18.75\% \rightarrow 37.5\% \rightarrow 56.25\% \rightarrow 50.0\%$. At every round, the test prompt contains only the bare question. `src/agenthn/webapp/skills_service_product.py`.

We deliberately report round 3's accuracy as the final number rather than the best-of-rounds (56.25% at round 2): a hill-climbing loop built from iterative rewriting and re-internalization is not guaranteed to be monotonic, paralleling known non-monotonicity in RL-style and self-refine fine-tuning loops, and reporting the raw trajectory (Figure 8) rather than cherry-picking the best checkpoint is the more defensible practice. The headline result is not the specific numbers but the mechanism: at every single round, the model that is tested has *nothing* in its prompt but the bare question – everything it has learned so far, including its own self-critique from the previous round, lives in the adapter's weights, and that adapter is replaced, not fine-tuned, every round.

9 Discussion

Across all four tracks, the same primitive – internalize a document into an adapter in one forward pass, then evict the document from the prompt – recurs with different documents: an evictable conversation segment (memory), a self-extracted preference profile (personalization), a

[2] https://github.com/obra/superpowers

reference doc or procedural file (skills), and an agent's own self-written, iteratively-revised notes (self-improvement). The one place this primitive needed augmentation rather than direct reuse is memory: D2L's native chunk-concatenation is built for one bounded document, and applying it naïvely to an open-ended stream produces destructive interference once enough segments accumulate (§5.2). The fix – retrieve before you compose – is simple, but the ablation in Table 1 shows it is not optional: every condition that skips it scores $0/6$, regardless of whether the underlying per-segment encoding is in principle capable of perfect recall.

The recurring trade we observe against text-based retrieval is consistent across both memory and skills: a $\sim$two-order-of-magnitude reduction in query-time context, in exchange for 5–13 points of recall, because the hypernetwork's encoding is lossy in a way that exact text retrieval is not. Whether that trade is worth it is a deployment decision, not a universal answer – it is attractive exactly when query-time context cost (latency, KV-cache memory, dollars) dominates the cost of being occasionally wrong, and a 2B base model's residual encoding capacity is the binding constraint on how small that "occasionally" can be made; we would expect the recall side of this trade to improve with a larger base model and a hypernetwork trained at matching scale, since D2L's own architecture does not depend on model size.

Limitations

Single small base model. Every result in this paper uses one 2B-parameter base model (`gemma-2-2b-it`) and one released D2L checkpoint. The 2B model's limited residual capacity is almost certainly the dominant source of the lossy-recall gap we observe (e.g. "Ayemi" vs. "Adeyemi," §5), and our skill-acquisition ceiling (55% on physics word problems) reflects the base model's reasoning ability as much as the internalization mechanism. We have not evaluated whether our findings – particularly the destructive-interference failure mode and the retrieval fix – hold at larger scale.

Lossy compaction is fundamental, not incidental. NAPLORA consistently trades recall for context cost relative to retrieving text; we report this honestly (§5.7, §5.8) rather than presenting only favorable comparisons, but the gap means NAPLORA is not a drop-in replacement for RAG wherever recall is the only thing that matters.

Estimated costs. Our Cartridges training-cost estimate (§5.8, $\sim$\$43 per corpus) is an explicitly order-of-magnitude, labeled estimate (modeled as $300\times$ the cost of one read of the corpus), not a directly measured wall-clock training bill; our Cartridges *recall* number, by contrast, is measured from an actual trained model and should not be conflated with the cost estimate's precision.

Synthetic evaluation. Our memory and self-improvement probes use synthetic, deliberately unguessable facts and a fictional product specifically so that no result can be attributed to the base model's pretraining priors about a real entity. This is the right design for isolating the mechanism, but it limits ecological validity relative to real, messy long-horizon agent transcripts, which we have not evaluated on.

Non-monotonic self-improvement. The self-improvement trajectory (§8) dips at round 3; we have no mechanism that guarantees monotonic improvement, and a practitioner deploying this loop would need an external stopping criterion (e.g. a held-out validation slice) rather than assuming more rounds are always better.

Checkpoint-specific multi-chunk support. The publicly released D2L checkpoints differ in whether multi-chunk rank-concatenation is in-distribution; using the single-chunk-trained checkpoint for memory composition silently produces degenerate adapters rather than raising an error, a sharp edge we encountered directly during development and flag for anyone reproducing this work.

Ethical Considerations

The personalization track stores user-specific preferences in model weights rather than in a plain-text profile, which could plausibly be read as more private (the profile text never sits in a server log of prompts) or less private (a weight delta is non-human-readable but not necessarily harder to extract via probing); we have not evaluated extraction attacks against internalized adapters and recommend treating an adapter as containing the same sensitive information as the document it was built from. All synthetic facts, the fictional "Lumen" product, and the held-out physics/formatting questions in this paper are fabricated for evaluation purposes and do not reference real individuals, companies, or proprietary material.

10 Conclusion

We presented AGENTHN, a framework for letting agents edit their own weights at inference time via the Doc-to-LoRA hypernetwork, and NAPLORA, a streaming memory mechanism that extends D2L's bounded single-document chunking into an unbounded conversational horizon by adding retrieval-gated composition – without which the natural extension fails completely via destructive interference. Across memory, personalization, skill acquisition, and self-improvement, internalizing a document into an adapter and evicting it from the prompt consistently trades a measurable, bounded amount of recall for an order-of-magnitude reduction in query-time context and cost, with no gradient training at inference time in any of the four tracks. We release all code, evaluation scripts, captured fixtures, and the exact numbers reported in every table and figure of this paper.

Acknowledgments

We thank Sakana AI for releasing Doc-to-LoRA's code and checkpoints, without which none of this work would have been possible, and the organizers of the hackathon this project began at.

References

Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, and 1 others. 2021. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*.

Rujikorn Charakorn, Edoardo Cetin, Yujin Tang, and Robert Tjarko Lange. 2025. Text-to-LoRA: Instant transformer adaption. In *Forty-second International Conference on Machine Learning (ICML)*.

Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. 2026. Doc-to-LoRA: Learning to instantly internalize contexts. *Preprint*, arXiv:2602.15902. Sakana AI.

Tong Chen, Hao Fang, Patrick Xia, Xiaodong Liu, Benjamin Van Durme, Luke Zettlemoyer, Jianfeng Gao, and Hao Cheng. 2025. Generative adapter: Contextualizing language models in parameters with a single forward pass. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. 2019. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of NAACL*.

Sabri Eyuboglu, Ryan Ehrlich, Simran Arora, Neel Guha, Dylan Zinsley, Emily Liu, Will Tennien, Atri Rudra, James Zou, Azalia Mirhoseini, and 1 others. 2025. Cartridges: Lightweight and general-purpose long context representations via self-study. *arXiv preprint arXiv:2506.06266*.

David Ha, Andrew Dai, and Quoc V. Le. 2016. Hypernetworks. *arXiv preprint arXiv:1609.09106*.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. 2024. RULER: What's the real context size of your long-context language models? In *First Conference on Language Modeling (COLM)*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Andrew Jaegle, Felix Gimeno, Andrew Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. 2021. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning (ICML)*, pages 4651–4664. PMLR.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.

Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597.

Yichuan Li, Xiyao Ma, Sixing Lu, Kyumin Lee, Xiaohu Liu, and Chenlei Guo. 2024. MEND: Meta demonstration distillation for efficient and effective in-context learning. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.

Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2024. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems*, 36.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Ruhle, Yuqing Yang, Chin-Yew Lin, and 1 others. 2024. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. *arXiv preprint arXiv:2403.12968*.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392.

Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.

Charlie V. Snell, Dan Klein, and Ruiqi Zhong. 2023. Learning by distilling context.

Johannes von Oswald, Christian Henning, Benjamin F. Grewe, and Joao Sacramento. 2020. Continual learning with hypernetworks. In *International Conference on Learning Representations (ICLR)*.

A Prompts

Personalization extractor (§6). The base model is prompted, after every user turn, with a fixed instruction to output zero or more lines of the form `ACTION | CATEGORY | VALUE` (`add`/`update`/`remove`; `NONE` if nothing durable is said), with two worked examples (a moving/profession update, a dietary-preference correction) included in the prompt to anchor the line format, since a 2B model is unreliable at emitting well-formed JSON for this task.

Self-improvement study/reflect prompts (§8). Round 1: *"Read the following product reference material and write concise study notes (about 5 sentences) capturing the most important facts a support agent would need."* Later rounds: the model is shown its current notes and the list of *questions* (not answers) it got wrong, and instructed to *"find the facts these questions ask about in the reference material and rewrite your study notes so you can answer them next time,"* explicitly keeping whatever in the old notes remains useful rather than discarding it.

Skill-router classifier (§7). *"Classify the question below into exactly one category: PHYSICS (a mechanics word problem...) or FORMATTING (a request to produce structured output like JSON, YAML, protobuf, or a bulleted list). Reply with one word."*

B Full statistical validation table

Size	NAPLORA	text-RAG	vanilla
2k	85.0 (7.2)	95.0 (116.1)	95.0 (1964.0)
8k	87.0 (7.2)	100.0 (115.8)	36.0 (7755.8)
16k	87.0 (7.2)	100.0 (115.8)	0.0 (8192.0)
32k	87.0 (7.2)	100.0 (115.8)	0.0 (8192.0)
48k	87.0 (7.2)	100.0 (115.8)	0.0 (8192.0)

Table 3: Pooled recall % (query-time context tokens), n=100 trials/cell (5 seeds $\times$ 20 facts). `results/large_scale.json`.

C Scaling sweep, full table

Target	Raw tok	Segs	NAPLORA	Vanilla
1k	1,025	10	6/6	6/6
2k	2,029	19	6/6	6/6
4k	4,004	36	6/6	6/6
8k	8,002	70	6/6	4/6
16k	16,009	138	6/6	0/6
32k	32,023	274	6/6	0/6
48k	48,006	410	6/6	0/6

Table 4: Six-needle recall as one conversation grows to 48k tokens. NAPLORA's query-time context stays at 9.5 tokens throughout (vs. vanilla growing to the 8,192-token window cap). `scripts/scaling_sweep.py`.

D Held-out physics word problems

Representative items from the 20-question battery (§7.1; full list in `skill-acquisition/golden-examples.txt`): *"A 15 kg box sits on a horizontal floor with a coefficient of friction of 0.3. What is the maximum friction force?"* (expected 44.1N); *"Two masses of 10 kg and 3 kg are 2 m apart. What is the gravitational force between them ($G = 6.674 \times 10^{-11}\,\mathrm{N\,m^2/kg^2}$)?"* (expected

5.0055×10^{-10}N); *"A 1000 kg car accelerates from rest to 20 m/s in 4 seconds. What net force acted on the car?"* (expected 5000N).

E Reproducibility

All numbers in this paper are produced by scripts committed in the project repository and read programmatically from their JSON outputs (`results/*.json`, `skill-acquisition/physics_benchmark_results.json`) rather than transcribed from figures. Base model: `google/gemma-2-2b-it` (gated) / `unsloth/gemma-2-2b-it` (ungated mirror, identical weights) throughout. Greedy decoding (`do_sample=False`) is used for every numeric result reported; sampling temperature is not a free parameter anywhere in this paper.